

מערכות קבצים

מערכות הפעלה

מערכות קבצים

בפרקים הקודמים למדנו על סוגי זיכרונות אך כשעשינו זאת מרבית העיסוק שלנו היה עם זיכרון ה-RAM. למדנו ש-RAM הוא זיכרון נדיף (Volatile) ומהיר יחסית, למדנו כיצד נראות כתובות וירטואליות, כיצד הן מומרות לכתובות פיזיות ב-RAM, למדנו על מנגנון ה-Paging וכיצד מבוצע Context Switch, למדנו כיצד קובץ EXE נטען ל-RAM.

לעומת זאת, לזיכרון בלתי נדיף (Non Volatile) כגון דיסק קשיח או התקן USB, הקדשנו עד כה תשומת לב מועטה. הנחנו שיש התקן זיכרון בגודל עצום, שהקבצים פשוט נשמרים בו ואנחנו קוראים וכותבים אליהם כרצוננו. אך כיצד הקבצים נשמרים? מאחורי השאלה הפשוטה הזו יש תהליך מורכב ויפה ועליו נענה בפרק זה.

מערכות קבצים

בתחילת הפרק נדון בשתי טכנולוגיות של שמירת מידע- דיסק מגנטי ותא זיכרון מסוג Flash. אם אתם משתמשים בדיסק קשיח מסוג HDD, דיסק קשיח מסוג SDD או התקן זיכרון מסוג USB שהשם השגור שלו הוא "Disk On Key", אתם עושים שימוש באחת הטכנולוגיות הללו.

לאחר סקירה קצרה של החומרה נדון בעקרונות הנדרשים למערכת קבצים. לא נסקור מערכת קבצים ספציפית, אך נבין מה הדברים שמערכת קבצים מתפקדת צריכה לכלול.

בהמשך הפרק נמחיש את העקרונות באמצעות מערכת קבצים שפותחה במיוחד לצורך פרק זה. מערכת הקבצים הינה דימוי של מערכת קבצים אמיתית. כדי לא להזדקק להתקנת חומרה חדשה, מערכת הקבצים יוצרת קובץ רגיל על הדיסק הקשיח שלכם ומעליו מנהלת שמירה ומחיקה של קבצים ותיקיות, ממש כאילו הקבצים שמורים על חומרה נפרדת.

מערכות קבצים

התקני זיכרון בלתי נדיף

יש סוגים רבים של התקני זיכרון בלתי נדיף. מרביתם היסטוריים: טייפ מגנטי, כרטיסיות נייר מנוקבות, דיסקטים (Floppy Disks). שמור להם מקום של כבוד אולם לא נעסוק בהם. התקני זיכרון נוספים שלא נעסוק הם CD ו-DVD, ראשית כיוון שגם הם מיושנים למדי ושנית כיוון שמרבית ההתקנים מסוגים אלו ישמשו לכתיבה חד פעמית ואילו אנחנו מתעניינים במערכות קבצים שמתעדכנות באופן שוטף בזמן פעולת המחשב.

שלושת ההתקנים הנפוצים כיום, סביר שאתם משתמשים בלפחות אחד מהם: דיסק קשיח מסוג HDD, התקן USB "Disk On Key", דיסק קשיח מסוג SSD. נסקור בקצרה את הטכנולוגיות שעומדות בבסיסם.

מערכות קבצים

אחסון מגנטי

דמיינו שיש בידיכם אוסף מגנטים, כמו שיש על מקרר ביתי לדוגמה. לכל מגנט יש שני צדדים, מקובל לקרוא להם "צפון" ו"דרום". בין שני צדדים הפוכים יש משיכה ובין שני צדדים זהים יש דחיה. כך, ה"צפון" נמשך ל"דרום", אולם אם תנסו לקרב שני צדדים מסוג זהה הכח המגנטי יפעל נגדכם וידחה את המגנטים זה מזה. אתם יכולים להניח את המגנטים על שולחן ולסובב כל מגנט בכיוון מסוים: לעיתים הצפון יהיה עם הפנים אליכם ולעיתים הדרום. נניח שהצד "דרום" משמעותו 0 והצד "צפון" משמעותו 1. כעת יש על השולחן לפניכם רצף של אחדות ואפסים מקודד באמצעות מגנטים. איך אפשר לקרוא את הרצף? לשם כך ניקח מגנט נוסף שהכיוון שלו ידוע, לדוגמה "צפון", ונעביר אותו על פני רצף המגנטים. בכל פעם שנחוש משיכה נדע כי אנחנו קוראים "דרום", כלומר 0. בכל פעם שנחוש דחיה נרשום 1. כך מתבצע פענוח המידע.

מערכות קבצים

אחסון מגנטי

זהו עקרון הפעולה של הדיסק הקשיח מסוג HDD. ישנו דיסק, או מספר דיסקים, המצופים בחומר מגנטי. ביט נשמר באמצעות שינוי נקודתי של החומר המגנטי על הדיסק כך שיהיה צפונה או דרומה, 0 או 1. ראש מגנטי עדין עובר על פני הדיסק וקורא את המידע. הראש המגנטי מסוגל גם לשמש לכתיבה, באמצעות הפעלת מגנט על החומר המגנטי ניתן לשנות את כיוון השדה שעל הדיסק.

מערכות קבצים

אחסון מגנטי

מהתיאור עד כה ברור כי צריך מנגנון מכני שיניח את הראש הקורא / כותב מעל המקום הנכון בדיסק. המנגנון הזה מורכב בפועל משני גורמים: רכיב שמניח את הזרוע על הרדיוס הנכון של הדיסק, ורכיב שמסובב את הדיסק כדי שהזרוע תהיה מעל הנקודה הנכונה.

המנגנון המכני, מתוחכם ככל שיהיה, עדיין סובל משתי בעיות.

הבעיה הראשונה היא איטיות יחסית. גם אם לוקח אלפית שניה בלבד להניח את הראש הקורא מעל המקום הנכון בדיסק, מבחינת שעון המעבד עבר זמן של כמיליון פעולות. כמעט נצח.

הבעיה השניה היא נטיה לשגיאות. חומר מגנטי עלול לשנות את כיוונו, במיוחד כשהטכנולוגיה היא כה צפופה וביט נשמר בשטח שגודלו אטומים בודדים.

מערכות קבצים

תא זיכרון מסוג Flash

השם Flash Memory ניטבע על ידי מהנדסי חברת Toshiba שעסקו בפיתוח הטכנולוגיה. מבלי להיכנס להסבר מעמיק על טכנולוגיית טרנזיסטורים, זיכרון ה-Flash עובד באופן הבא: תא הזיכרון מכיל "כספת" שיכולים להיות בתוכה אלקטרונים. ליד הכספת יש חומר מוליך, שיכול להיות מקור לאלקטרונים או לאסוף אלקטרונים אליו. אלקטרונים לא יכולים להיכנס לכספת ולצאת ממנה סתם כך, אך אם יופעל מתח חזק מספיק לפרק זמן מספיק, האלקטרונים יצליחו לפרוץ את קירות הכספת ולהיכנס או לצאת ממנה, כתלות בכיוון המתח שיופעל. כך נוצר מנגנון שמאפשר לשמור אלקטרונים ולהחזיק אותם שם לנצח, בלי שצריך לשמור על מקור מתח קבוע: מרגע שהפעלנו מתח וקבענו את מצב הכספת, המצב יישמר. כדי לבדוק אם יש או אין אלקטרונים בכספת נשתמש במתח נוסף, חלש יותר. רמת הזרם שנמדדת מושפעת מהאלקטרונים שנמצאים בכספת. יש זרם- הכספת ריקה. אין זרם- הכספת מלאה. כלומר הצלחנו לשמור אפס או אחד ולקרוא אותם כרצוננו.

מערכות קבצים

תא זיכרון מסוג Flash

אפשר להשיג שיפור אם משתמשים במספר רמות זרם. אם במקום לקבל "יש זרם" או "אין זרם" אפשר לקבל מדידה מדוייקת של הזרם אז אפשר לחלק את עוצמת הזרם לרמות שונות כאשר כל רמה מייצגת צירוף אחר של אחדות ואפסים. ארבע רמות של זרם מאפשרות לייצג שני ביטים בתא זיכרון יחיד, שמונה רמות של זרם מאפשרות לייצג שלושה ביטים וכך הלאה.

טכנולוגיית ה-Flash היא הבסיס לפעולת ה-Disk On Key והדיסק הקשיח מסוג SSD.

Disk-on-Key הוא התקן זיכרון חיצוני בלתי נדיף שניתן לחבר לכל כניסת USB ולהשתמש בו כדיסק חיצוני. רבים מכנים את התקן הזיכרון "USB Stick" אך חשוב לזכור שטכנולוגיית ה-USB היא רק הממשק שדרכו מתחברים לזיכרון. השם Disk On Key הומצא על ידי חברת M-Systems הישראלית, שהיתה הראשונה לקחת זיכרון Flash ולחבר אליו ממשק USB שיאפשר חיבור מיידי למחשב.

מערכות קבצים

תא זיכרון מסוג Flash

דיסק קשיח מסוג SSD, קיצור של Solid State Drive, הוא התקן שאין בו חלקים מכניים נעים, בניגוד לדיסק HDD שסקרנו לפני כן. טכנולוגיית ה-Flash נפוצה כיום ב-SSDים, בדומה ל-Disk On Key, אך אין הדבר אומר ש-SSD חייב להיות בטכנולוגיית Flash. ישנם SSDים מהירים שכוללים זיכרון זהה לזיכרון המהיר של ה-RAM, החיסרון שלהם הוא שהם חייבים לקבל זרם באופן קבוע, עם הפסקת הזרם המידע שעליהם יימחק.

ל-SSD יתרונות רבים על HDD: העובדה שאין בו חלקים נעים מאפשרת פעולה שקטה יותר. מהירות הקריאה והכתיבה מהדיסק ואליו עולות, מכיוון שאין מגבלות מכניות על מהירות הסיבוב של הדיסקים כמו ב-HDD. כתוצאה מכך טעינת תוכנות גדולות מתבצעת במהירות רבה יותר מ-SSD.

מצד שני, תאי הזיכרון מסוג Flash אינם שומרים על המידע לאורך זמן כמו התקן מגנטי, ופעולות רבות של כתיבה מקצרות את תוחלת החיים שלהם. לכן לשמירת נתונים לטווח זמן ארוך HDD עדיין טכנולוגיה עדיפה.

מערכות קבצים

Sector, Physical Sector, Cluster, Page, Block

נסקור יחידות מידה של זיכרון.

יחידת המידה הבסיסית של זיכרון על דיסק HDD הינה Sector, ואילו יחידת המידה הבסיסית של זיכרון על דיסק SSD נקראת Block. זהו אוסף של בתים שנמצאים פיזית זה לצד זה ויש להם הגנה של קוד לתיקון שגיאות. קוד לתיקון שגיאות הינו בתים נוספים שנמצאים ב-Sector ומסייעים לבדוק אם היו שגיאות ובמידת הצורך גם לתקן אותן. עוד על קוד לתיקון שגיאות אתם מוזמנים לקרוא בהרחבה בהמשך.

מערכות קבצים

Sector, Physical Sector, Cluster, Page, Block

תוספת הבתים לטובת תיקון השגיאות משמעותה שלא כל הבתים ב-Sector משמשים לשמירת מידע, לכן יש נטייה בשנים האחרונות להגדיל את גודל ה-Sector ולשפר את הקוד לתיקון שגיאות כדי שתידרש תוספת בתים קטנה יותר כדי להגן על המידע. בדרך כלל גודלו של Sector הוא 512 בתים. ייתכן שאתם שמים לב שכבר נתקלנו בעבר בגודל של 512 בתים, או 0x200. היזכרו בפרק על פורמט PE, זהו בדיוק הערך של שדה ה-File Granularity שבתוך ה-Optional Header. בעבר אכן כל קובץ היה צריך להיות בגודל שהוא כפולה שלמה של Sector, מיד נראה שכיום גודל של קובץ צריך להיות כפולה שלמה של מספר Sector'ים.

מערכות קבצים

Sector, Physical Sector, Cluster, Page, Block

Physical Sector הינה יחידה המידה הקטנה ביותר שהמנגנון הפיזי של הדיסק מסוגל לגשת אליה. ייתכן שהיא כוללת Sector יחיד וייתכן שהיא כוללת מספר Sectors. כלומר באופן פיזי החלקים שמצבעים פעולות כתיבה וקריאה בדיסק לא מסוגלים לבצע פעולת קריאה וכתיבה על משהו שגודלו קטן מגודל Physical Sector.

היחידה הבאה היא Cluster, או Logical Sector. בזיכרון SSD היא נקראת Page. זו יחידת המידה הקטנה ביותר שמשמשת לשמירת קבצים והיא יכולה לכלול מספר Physical Sectors או אחד בלבד. במילים אחרות, מי שמסתכל על הדיסק מהכיוון של מערכת ההפעלה לא מודע לכך שיש יחידת מידה בשם Sector או Physical Sector, המנגנון הפיזי נסתר ממערכת ההפעלה. מערכת ההפעלה מכירה את ה-Cluster, לכן כל גודל של קובץ תופס על הדיסק כפולה שלמה של Clusters.

מערכות קבצים

Sector, Physical Sector, Cluster, Page, Block

לדוגמה, אם גודלו של Cluster מוגדר להיות 1024 בתים ואנחנו מעוניינים לקרוא את ערכו של הבית במקום השלישי ולהוסיף לו 1, אין לנו יכולת לקרוא רק את הבית השלישי ולאחר מכן לכתוב רק לתוכו את הערך העדכני. נצטרך לקרוא את כל 1024 הבתים של ה-Cluster לתוך זיכרון ה-RAM שלנו, לבודד מתוך המידע את הבית השלישי, לשנות את ערכו ולאחר מכן לכתוב את כל 1024 הבתים של ה-Cluster בחזרה מה-RAM אל הדיסק הקשיח.

מערכות קבצים

Sector, Physical Sector, Cluster, Page, Block

לכאורה זו נראית שיטה בזבזנית ביותר. מה היתרון שלה?

כאשר נדרשת גישה לבית מסוים בדיסק הקשיח, סביר מאד שגם הבתים הקרובים אליו נדרשים. לדוגמה, אם ה-Loader צריך לטעון קובץ הרצה ל-RAM, לא נדרש רק בית יחיד אלא אוסף של בתים קרובים. נניח שיצרנו קובץ הרצה ואנחנו שומרים אותו לדיסק הקשיח. כיוון שתהליך הכתיבה נעשה גם הוא לתוך Cluster-ים, הרי שמובטח לנו שכל אחד מה-Cluster-ים שהשתמשנו בהם יכיל מידע ששייך רק לקובץ ששמרנו לדיסק.

מערכות קבצים

Sector, Physical Sector, Cluster, Page, Block

נניח שהקובץ שלנו בגודל של 2200 בתים ושגודל Cluster הוא 1024 בתים. נצטרך שלושה Cluster'ים: ה-Cluster הראשון יכיל 1024 בתים

ה-Cluster השני יכיל גם הוא 1024 בתים

ה-Cluster השלישי יכיל את יתרת הבתים בקובץ, 2200 פחות 2×1024 , כלומר 152 בתים בלבד. כפי שאנחנו רואים יש לנו בזבז של מקום בסוף ה-Cluster השלישי, מנוצלים פחות בתים ממה שה-Cluster מסוגל לשמור.

נראה שהכתיבה בזבזנית מבחינת הרווחים הלא מנוצלים שנותרים בדיסק, אך לעומת זאת יש בקריאה יתרון משמעותי: כדי לקרוא את כל הקובץ מספיקות רק 3 פעולות קריאה. כך, זמן הקריאה, שאמרנו שהוא גדול מאד יחסית למהירות המעבד, מתחלק בין כל הבתים שב-Cluster.

מערכות קבצים

נותר לנו לדון במושג ה-Block. מושג זה שונה בין HDD ל-SSD. בעוד ב-HDD הבלוק משמש לעיתים כמושג חלופי ל-Physical Sector, ב-SSD הכוונה לצבר של Pages, וזוהי היחידה הקטנה ביותר של זיכרון שניתן למחוק. המשמעות היא שבעוד קריאה מ-SSD מתבצעת בכפולות של Page, הכתיבה עלולה להיות יקרה: אם רוצים לשנות משהו ב-Page קיים, צריך קודם כל למחוק את המידע הקיים. כאמור כדי למחוק משהו מה-Page צריך למחוק את כל ה-Block שה-Page נמצא בתוכו. המסקנה היא שאם אין Block פנוי, פעולת הכתיבה לדיסק הופכת להיות יקרה ביותר ו-SSDים הופכים להיות יותר איטיים ככל שהם מתמלאים וכמות ה-Blockים הריקים בהם קטנה.

מערכות קבצים

מהו גודל ה-Sector במחשב שלכם? וה-Cluster?

פיתחו cmd בהרשאת admin (חפשו cmd בשורת חיפוש של Windows ואז בחרו להריץ כמנהל). הקלידו:

```
fsutil fsinfo ntfsinfo c:
```

קבלתם את הגדלים השונים של ה-Sector, Physical Sector, Cluster. בדוגמה זו, ה-Cluster כולל Physical Sector יחיד.

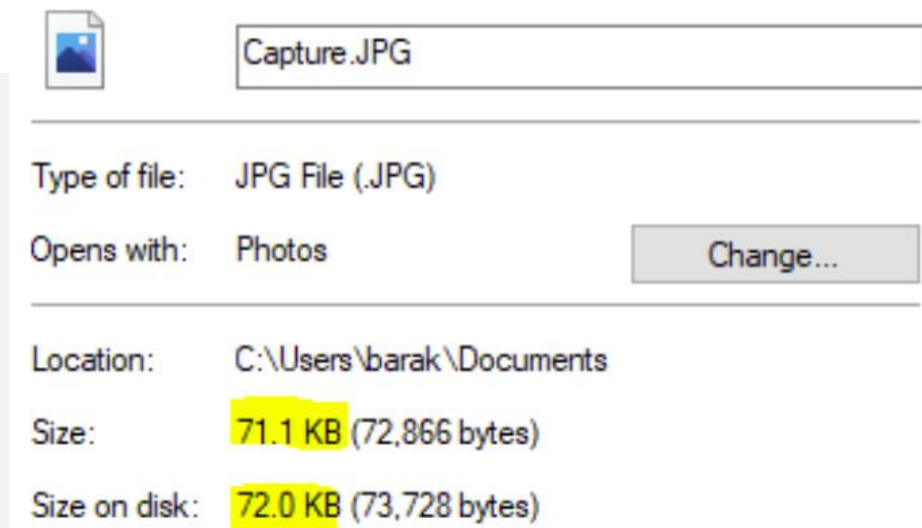
```
C:\WINDOWS\system32>fsutil fsinfo ntfsinfo c:
NTFS Volume Serial Number :           0x76aae7f9aae7b431
NTFS Version                   :           3.1
LFS Version                   :           2.0
Total Sectors                 :      945,170,431   (450.7 GB)
Total Clusters                :      118,146,303   (450.7 GB)
Free Clusters                 :       36,112,907   (137.8 GB)
Total Reserved Clusters      :          6,292    ( 24.6 MB)
Reserved For Storage Reserve :              0    (  0.0 KB)
Bytes Per Sector              :           512
Bytes Per Physical Sector     :          4096
Bytes Per Cluster             :          4096
Bytes Per FileRecord Segment  :          1024
Clusters Per FileRecord Segment :           0
Mft Valid Data Length         :          815.75 MB
Mft Start Lcn                 :      0x000000000000c000
Mft2 Start Lcn                :      0x0000000000000002
Mft Zone Start                :      0x00000000051fdc00
Mft Zone End                  :      0x000000000520a2e0
MFT Zone Size                 :          198.88 MB
Max Device Trim Extent Count  :           256
Max Device Trim Byte Count    :      0xffffffff
Max Volume Trim Extent Count  :            62
Max Volume Trim Byte Count    :      0x40000000
Resource Manager Identifier    :      249A32D0-32B1-11E8-8F8C-00809B0043FF
```

מערכות קבצים

כעת נבדוק אם מה שקבלנו הגיוני. אנחנו מצפים שגודלו של קובץ על הדיסק יעוגל כלפי מעלה להיות כפולה

שלמה של גודל ה-Cluster, במקרה זה 4096 בתים. Bytes Per Cluster : 4096

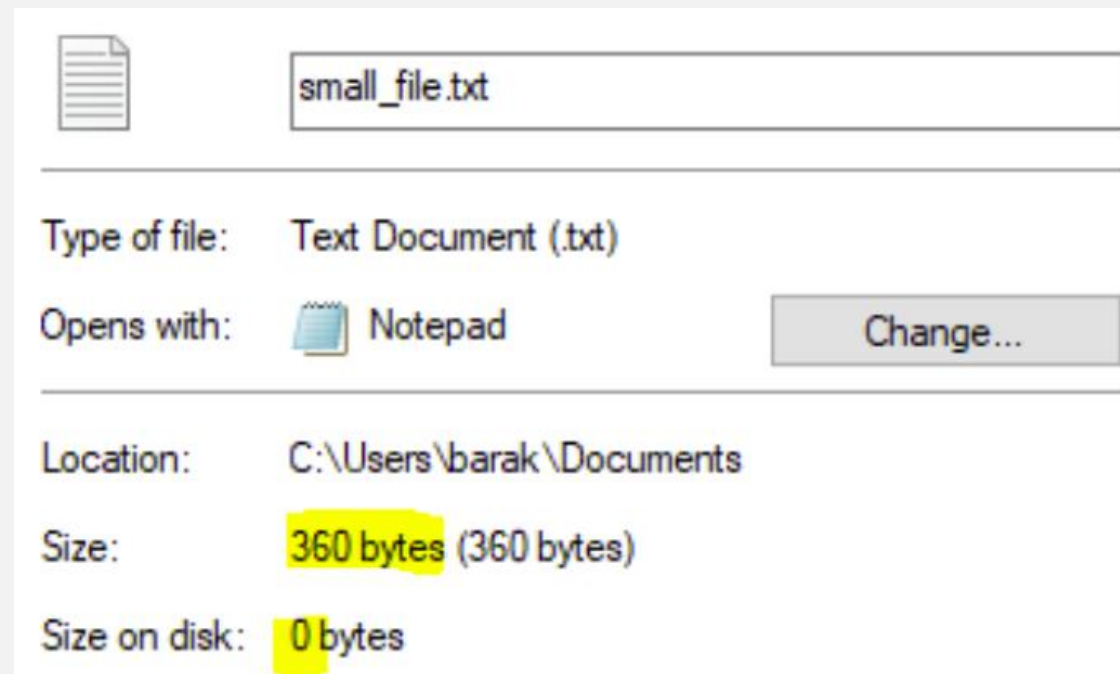
גשו ל-properties של קובץ כלשהו, מיצאו את גודל הקובץ על הדיסק- האם קבלתם מספר שהוא כפולה של גודל ה-Cluster מעוגלת כלפי מעלה? לדוגמה-



גודלו של הקובץ הינו 72,866 בתים. עם זאת, על הדיסק מוקצים לו 72,728 בתים, שהם בדיוק 18 פעמים גודל ה-Cluster של 4096 בתים.

מערכות קבצים

יכול להיות אולי שבחרתם קובץ קטן למדי, במקרה זה אתם עשויים לקבל תוצאה מפתיעה, לדוגמה:



האם אכן גודל הקובץ הוא 0 בתים? כלל וכלל לא. אלא שהקובץ הזה אינו שמור ב-Cluster רגיל על הדיסק, היכן שנשמר המידע. עבור קבצים קטנים במיוחד, מתחת ל-1KB, המידע עצמו נשמר יחד עם המידע שמאפשר למערכת ההפעלה למצוא את הקובץ. נבין זאת בהמשך, כשנחקור כיצד מערכת קבצים מאורגנת.

מערכות קבצים

קוד לתיקון שגיאות

הרעיון של קוד לתיקון שגיאות הוא פשוט- נוסף מידע שהוא חישוב מתמטי כלשהו על המידע שקיים, המידע הנוסף יאפשר לנו לא רק לדעת שנפלה שגיאה אלא גם לתקן אותה בסבירות גבוהה.

מערכות קבצים

לדוגמה, יש לנו ארבעה ביטים, 0111, שאנחנו מעוניינים להגן עליהם מפני שגיאות. נסדר אותם בריבוע:

0	1
1	1

לכל עמודה ולכל שורה נוסיף ביט נוסף, שהוא תוצאה של or בין הביטים בעמודה או בשורה:

0	1	1
1	1	0
1	0	

לכן נכתוב לדיסק לא רק 0111 אלא 0111 1010, כאשר ארבעת הביטים הראשונים הם המידע וארבעת האחרונים הם הקוד לתיקון שגיאות.

מערכות קבצים

0	0	1
1	1	0
1	0	

נבדוק מה קורה אם נפלה שגיאה, לדוגמה קראנו 1010 0011:

התוצאה של ביצוע ה-xor לא מתיישבת עם קוד התיקון השגיאות של השורה הראשונה. כך אנחנו יודעים שחל שיבוש בשורה הראשונה. אך היכן בדיוק השיבוש, באיזה משני הביטים בשורה הראשונה?

לאחר שנבצע xor גם על העמודות נמצא כי אחת העמודות תקינה ואחת לא. כעת אנחנו יכולים להצליב את השורה הלא תקינה עם העמודה הלא תקינה, לקבל את המקום המדויק של השגיאה ולדעת מה צריך לתקן כדי לקבל את המידע המקורי.

מערכות קבצים

כמובן שהמחשת הקוד לתיקון שגיאות שסקרנו כאן היא בזבזנית, על כל ביט של מידע נדרש ביט של קוד לתיקון שגיאות. אפשר להפוך את הקוד ליעיל יותר אם משתמשים במטריצה גדולה יותר, לדוגמה אם נשתמש במטריצה של 5×5 על כל 25 ביטים של מידע יהיו לנו רק 10 ביטים של תיקון שגיאות. החיסרון הוא שככל שהמטריצה יותר גדולה, כך גדל הסיכון שהיא תכלול מספר שגיאות ואז מנגנון תיקון השגיאות עלול להיכשל.

דיסקים משתמשים בקוד שנקרא Reed Solomon. זהו קוד שהחזקה שלו היא היכולת לתקן רצף שגיאות צמודות זו לזו. הסיבה לכך היא שבדרך כלל שגיאות בדיסקים לא יופיעו מפוזרות באופן אקראי אלא יופיעו במרוכז- לדוגמה עקב שריטה. קוד Reed Solomon "מערבב" ביטים מאזורים שונים כדי ליצור הגנה מפני שגיאות שמופיעות באופן מרוכז.

מערכות קבצים

עקרונות של מערכות קבצים

כדי להבין את עקרונות מערכת הקבצים, נתחיל מדף חלק ושלב שלב נוסיף עוד מורכבות.

שמירת מידע באופן טורי

כאמור אנחנו מתחילים מדף חלק. לרשותנו התקן חומרתי שמאפשר לנו לכתוב ולקרוא אחדות ואפסים. אנחנו צריכים להפוך את ההתקן הזה לכלי שמאפשר טיפול בקבצים- כתיבה, קריאה, מחיקה.

מערכות קבצים

נתחיל מפעולת הכתיבה לדיסק. נראה שהאלגוריתם הפשוט ביותר הוא "שמור את הקובץ במקום הריק הראשון שאתה מוצא". כך, אם נשמור לדוגמה שלושה קבצים כל קובץ יישמר במקום שבו הקובץ הקודם הסתיים. כך:

File 1

File 2

File 3

Empty

מערכות קבצים

טיפולנו בפעולת הכתיבה לדיסק. מה לגבי פעולת הקריאה? פעולה זו צריכה לקבל שם של קובץ ולהחזיר לנו את המידע שיש בו. קל מאד למצוא את הקובץ הראשון, בנמצא בתחילת הדיסק, אך איך נמצא את הקבצים הבאים? נצטרך ליצור לעצמנו קטלוג, כמו קטלוג של ספרן שאומר על איזה מדף נמצא כל ספר. קטלוג כזה צריך לכלול לכל הפחות שלושה דברים- שם הקובץ, המיקום שלו על הדיסק, וגודל הקובץ (כדי שלא נמשיך הלאה אל הקובץ הבא). כך, כאשר נבקש לקרוא קובץ מהדיסק תתבצע פעולת חיפוש בקטלוג, שתחזיר את מיקום תחילת הקובץ (אם הוא נמצא), או הודעת שגיאה (אם הוא אינו בקטלוג).

"Catalog"

File 1

File 2

File 3

Empty

מערכות קבצים

החלטנו למחוק קובץ כלשהו. חישבו- מה צריך להתבצע כדי למחוק את הקובץ?

ובכן, אפשרות אחת למחוק את הקובץ היא לגשת למיקום בו הוא שמור בדיסק ולכתוב אחדות ואפסים במקום המידע ששייך לקובץ. אולם, למעשה כל מה שצריך לעשות כדי שמערכת ההפעלה לא תמצא את הקובץ הוא פשוט לשנות את הרישום בקטלוג! מחיקת הרישום "שם קובץ- מיקום-גודל" בקטלוג מונעת ממערכת ההפעלה להחזיר לנו את הקובץ כיוון שאינה יודעת את מיקומו. זוהי שיטה מהירה ופשוטה, שמאפשרת דבר נוסף- שחזור של הקובץ המחק. הרי כל מה שצריך לעשות הוא לגשת לקטלוג ולהוסיף אליו מחדש את שם הקובץ והמיקום בדיסק, המידע של הקובץ נותר כשהיה. בהמשך נתנסה בשחזור מידע מחוק, כביכול.

מערכות קבצים

פרגמנטציה (שיברור)

שימו לב למקרה הבא. הדיסק שלנו כלל שלושה קבצים, אולם אז מחקנו את הקובץ האמצעי. כעת אנחנו מבקשים לשמור על הדיסק קובץ אחר, גדול יותר מהקובץ שמחקנו, ולמרות שיש מספיק מקום ריק בדיסק, אין לקובץ שלנו מקום. מה ניתן לעשות?

אפשרות אחת היא ליצור לעצמנו מקום ריק רציף, באמצעות העברה של הקובץ השני לסוף הקובץ הראשון. למרות שזו אפשרות ברת ביצוע, היא תעלה לנו במחיר יקר: העתקת הקובץ למקום אחר כדי לפנות מקום תיקח זמן.

מערכות קבצים

אפשרות שניה, היא לחלק את הקובץ לחלקים- כל חלק של הקובץ יישמר במקום ריק בגודל מתאים. שמירת חלקי קובץ נקראת פרגמנטציה, מהמילה "פרגמנט", ובעברית "שיברור". נצרך ידע קודם שלנו. גודל החלקים אינו יכול להיות כל גודל שמערכת ההפעלה יכולה לרצות, אלא חייב להיות כפולה של הגודל המינימלי שמערכת ההפעלה יכולה לבקש לשמור. אפשר להמשיל את הדבר לדף נייר. מערכת ההפעלה לא מקבלת דף נייר חלק שהיא יכולה לשים את חלקי הקובץ בכל מקום על הדף, אלא מקבלת דף נייר משבצות, שבו היא יכולה לבחור אם לתפוס משבצת או לא לתפוס משבצת. ייתכן שיידרשו מספר משבצות כדי לשמור קובץ מסויים, אך בשום מקרה שמירה לא תתחיל מאמצע משבצת אלא רק מתחילת משבצת וכאשר משבצת תפוסה היא תהיה תפוסה כולה.

מערכות קבצים

לכן, מחלקים את מערכת הקבצים לבלוקים בגודל קבוע, וכל בלוק שומר חלק מקובץ. אם גודל הקובץ גדול מגודל של בלוק, נשמור את הקובץ על פני מספר בלוקים.

"Catalog"	File 1a	File 1b	File 4a	File 3a	File 3b	File 4b
Block	Block	Block	Block	Block	Block	Block

כיצד צריכה להתבצע תהליך קריאת הקובץ מהדיסק? אנחנו כבר לא יכולים פשוט לגשת לתחילת הקובץ, כיוון שיש לו מספר התחלות. חישבו- איך אפשר לפתור את הבעיה?

מערכות קבצים

גם כאן יש יותר מפתרון אחד. אפשר שבסוף כל בלוק של קובץ יהיה מידע היכן נמצא המשך הקובץ. הבעיה בגישה זו, היא שנהיה מורכב להחזיק תמונת מצב של אילו בלוקים תפוסים ואילו ריקים בזיכרון. אנחנו רוצים לתפוס בלוק כלשהו בדיסק כדי לשמור קובץ חדש, כדי לדעת אם המקום פנוי יש צורך לעבור על כל הקבצים השמורים מתחילתם, ולבדוק בסוף כל בלוק אם הבלוק הבא נופל על האזור שאנחנו רוצים לדעת אם הוא פנוי.

אפשרות אחרת היא לשדרג את הקטלוג שלנו: כעת לכל קובץ תישמר רשימה שכוללת את מיקומי ההתחלה של כל הבלוקים ואת גודלם. לדוגמה, הקובץ hello.txt עשוי להתחיל מכתובת 0x2000 בדיסק ולתפוס 0x1000 בתים שם, להמשיך לכתובת 0x2000 ולתפוס שם שלושה בלוקים שמסתכמים ב- 0x3000 בתים, ולסיים בבלוק שמתחיל בכתובת 0x13000 וגודלו 0x1000 בתים. היתרון בגישה זו הוא שכדי לדעת אם אזור מסוים בדיסק תפוס, אנחנו יכולים להסתפק במעבר על הקטלוג.

מערכות קבצים

אם נסכם את מה שהבנו עד כה:

- א. לכל קובץ ששמור בדיסק צריכה להיות רשומה שבה כתוב היכן בדיסק שמורים החלקים השונים שלו. מרבית מערכות הקבצים קוראות לרשומה זו inode, קיצור של index node.
- ב. ישנו מקום מרכזי, קטלוג, שבו כל רשומות ה-inode צריכות להשמר. מקום זה נקראה inode table.
- ג. מרחב הכתובות של התקן הזיכרון מחולק כמו דף משבצות. מעתה נקרא לכל משבצת בשם בלוק.
- ד. כדי לדעת היכן ניתן לשמור קובץ חדש, יש צורך במקום שבו מרוכז הסטטוס של כל הבלוקים, מי תפוס ומי פנוי. המידע הזה מרוכז בטבלה בשם block table.

כעת לאחר שהבנו את העקרונות הללו נוכל לדון לפרטים כיצד בנויה מערכת קבצים פשוטה.

מערכות קבצים

מערכת קבצים Easy File System

מערכת הקבצים בה נדון כעת היא דמיונית, אך היא מבוססת רבות על מערכת הקבצים EXT2 של Linux.

מערכת הקבצים שלנו תישמר על קובץ בדיסק הקשיח, פשוט כדי לנצל את ההתקן הפיזי שכבר קיים. איננו מעוניינים בפיתוח ממשק פיזי חדש, אלא רק בהבנה של הארגון של מערכת הקבצים. מערכת הקבצים הפשוטה שלנו תשתמש בקובץ שגודלו 1MB.

נחלק את הקובץ לבלוקים בגודל 1K, 1024 בתים, כל אחד. כלומר יש לנו 1024 בלוקים שגודל כל בלוק 1024 בתים.

הבלוקים שמספרם 8 עד 1023 (הבלוק האחרון) ישמשו לשמירת המידע עצמו. הבלוקים שמספרם 0 עד 7 ישמשו לשמירת המידע הנלווה שנדרש כדי שמערכת הקבצים תוכל לבצע את תפקידיה, לשמור קבצים ולקרוא קבצים שמורים.

מערכות קבצים

SuperBlock

הבלוק הראשון (אינדקס 0) מכיל שלושה נתונים חשובים:

א. רצף ידוע מראש, שמסמן את תחילתה של מערכת הקבצים. בתהליך שבו מחשב נדלק ומערכת ההפעלה עצמה צריכה להיטען, מה שנקרא תהליך boot, התקן הזיכרון הפיזי מכיל קוד שטוען את מערכת ההפעלה. אחת הטכניקות שמשמשות מחשבים למציאת הקוד היא באמצעות סימון רצף ידוע מראש של אחדות ואפסים. במקרה שלנו, מערכת הקבצים מכילה בתחילתה את התווים EFS, קיצור של Easy File System.

ב. גודל של בלוק. מערכת ההפעלה צריכה לדעת מה גודלו של כל בלוק שמערכת הקבצים משתמשת בו, כיוון שהמספר הזה הכרחי לחישובי המיקומים השונים. לדוגמה, נניח שישנו קובץ שנמצא בבלוק מספר 12. המיקום הפיזי של הקובץ על הדיסק יחושב לפי 12 כפול גודל הבלוק. ללא ידיעת גודל הבלוק, מערכת ההפעלה לא יכולה להתמצא במערכת הקבצים. במקרה שלנו, גודל של בלוק נקבע להיות 1024 בתים.

מערכות קבצים

SuperBlock

הבלוק הראשון (אינדקס 0) מכיל שלושה נתונים חשובים:

ג. כמות הבלוקים במערכת הקבצים. מערכת ההפעלה חייבת לדעת כמה בלוקים יש במערכת הקבצים כדי להיות מסוגלת לשמור קבצים בצורה נכונה. אם מערכת ההפעלה לא מודעת לכך שקיימים, לדוגמה, רק 256 בלוקים, היא עלולה לבקש מההתקן הפיזי לשמור מידע בבלוק ה-257. במקרה שלנו, ישנם 1024 בלוקים.

גודלה של מערכת הקבצים שווה לכמות הבלוקים כפול גודלו של כל בלוק, לכן קבלנו מערכת בגודל 1MB.

מערכות קבצים

SuperBlock- ביצוע Format

הדבר הראשון שברצוננו לבצע הוא format למערכת הקבצים, מכיוון שלפני כן מערכת ההפעלה לא יודעת לזהות את מערכת הקבצים. בתהליך ביצוע format מתבצעים שלושה דברים:

1. נכתב ל-superblock המידע הנדרש כדי שמערכת ההפעלה תוכל לזהות את מערכת הקבצים ולעבוד מולה

2. מערכת הקבצים מתחילה מ"דף חלק", כך שהיא מציגה למערכת ההפעלה תמונה שבה אין שום קובץ או תיקיה ששמורים במערכת. שימו לב לניסוח- מוצג למערכת ההפעלה כאילו שאין שום קובץ או תיקיה ששמורים במערכת הקבצים. אין זה אומר שכל המידע נמחק, אלא כפי שנראה בהמשך- שדות שונים שמאפשרים את מציאת המידע מציגים לנו שאין מידע

מערכות קבצים

SuperBlock- ביצוע Format

3. מבוצע אתחול של תיקיית ה-Root. כפי שראינו, לכל תיקיה מוגדר מצביע לתיקיה ומצביע לתיקיה האם. בתהליך האתחול מערכת הקבצים שלנו תיצור את המצביעים המתאימים. בהמשך כאשר נדון בתיקייה Root נבין כיצד בדיוק הם שמורים.

מערכות קבצים

ה-Inode

עד כה הבנו, שכל קובץ נשמר בדיסק כשהוא fragmented: מחולק לחלקים, ששמורים במקומות שונים על הדיסק. הבנו גם שנחוץ קטלוג בשביל לשמור את המידע על כל קובץ.

לקטלוג הזה יש שם - inode, קיצור של index node. ה-inode הוא מבנה נתונים ששומר את מספרי הבלוקים בהם הקובץ שמור והגודל הכללי של הקובץ. בנוסף, יכולים להשמר נתונים נוספים, כגון האם הקובץ מיועד רק לקריאה או גם לכתיבה, מי הבעלים של הקובץ, תאריך היצירה של הקובץ ועוד. במקרה שלנו, נשמר גם שדה שמזהה האם מדובר בקובץ או בתיקיה (על תיקיות נדון בהמשך). אם כך, inode של EFS נראה כך:

מערכות קבצים

ה-Inode

- שדה מזהה סוג (בגודל של 4 בתים) – יקבל את הערך 1 אם מדובר בקובץ, 2 אם מדובר בתיקה, 0 אם הקובץ/תיקה מחוקים
 - שדה גודל הקובץ (בגודל של 4 בתים) – גודל הקובץ בבתים
 - סט של שישה מצביעים על בלוקים (בגודל של 4 בתים כל אחד) – כל מצביע הוא מספר בלוק שמשמש לשמירת המידע שנמצא בקובץ / תיקיה
- סך הכל גודלו של inode ב-EFS הוא 32 בתים.

מערכות קבצים

ה-Inode Table

טבלת ה-inodes היא המקום בו נשמרים זה אחרי זה כל ה-inodes שישנם במערכת הקבצים. במערכת הקבצים הקטנה שלנו EFS ישנם שישה בלוקים שמשמשים את ה-inode table. ב-EFS ה-inode table נשמר מיד לאחר ה-block table, כלומר בבלוקים מספר 2 עד 7.

מה הכמות המקסימלית של הקבצים שמערכת הקבצים שלנו יכולה לשמור?

כפי שראינו, גודלו של כל inode של EFS הוא 32 בתים. לכן כל בלוק שנקצה ל-inode table יכול להחזיק 32 inodes (1024, שהוא גודל הבלוק, חלקי 32).

כיוון שיש לנו 6 בלוקים ב-inode table, נוכל להחזיק לכל היותר 192 inodes.

כיוון שכל קובץ דורש inode, מערכת הקבצים שלנו יכולה לשמור לכל היותר 192 קבצים.

מערכות קבצים

על כמה בלוקים, לכל היותר, של מידע יוכלו להצביע ה-inodes שלנו?

כל inode כולל 6 מצביעים לבלוקים של מידע. לכן לכל היותר נוכל להצביע באופן ישיר על 1152 בלוקים. עם זאת, כיוון שמערכת הקבצים שלנו כוללת רק 1024 בלוקים, כמות הבלוקים הפנויים תתאפס לפני שטבלת ה-inodes תתמלא לגמרי. במילים אחרות, גם כשמערכת הקבצים מלאה לגמרי עדיין עשויים להיות inodes ריקים או כאלה שאינם מנצלים את כל 6 הבלוקים שאפשר להקצות לשמירת קובץ.

ה-inodes בטבלת ה-inodes מסודרים בזה אחר זה, וידועה לנו כתובת ההתחלה של הטבלה. לכן, כדי להגיע ל-inode מסויים כל מה שצריך לעשות הוא לקחת את האינדקס של ה-inode שאנחנו מחפשים, לכפול בגודל של inode ולהוסיף למקום בו טבלת ה-inodes מתחילה.

לדוגמה, במערכת הקבצים EFS, היכן נמצא ה-inode בעל אינדקס 1?

תשובה: כתובת ההתחלה של טבלת ה-inodes היא 0x800. גודל של inode הוא 32 בתים. לכן ה-node שהאינדקס שלו הוא 1 יחל בכתובת 0x820.

מערכות קבצים

Root ו- Data Blocks

ראינו שכל inode מכיל מצביעים לבלוקים שבהם מאוחסן המידע עצמו. כל הבלוקים החל מהבלוק שהאינדקס שלו הוא 8 ועד הבלוק האחרון, שהאינדקס שלו הוא 1023, הם בלוקים שמכילים את תוכן הקבצים והתיקיות. מספר הבלוק של ה-root ידוע מראש למערכת ההפעלה, כיוון שהוא הבסיס לכל פעולות הקריאה והכתיבה.

ה-root הינו תיקיה. כל תיקיה מחולקת לרשומות בגודל 32 בתים, כאשר כל רשומה יכולה להתייחס לקובץ או לתיקיה. המבנה הוא:

- בית 0: האם קיים (1- קיים, 0- מחוק)
- בתים 1 עד 27: שם הרשומה (הקובץ או התיקיה), מסתיים בסימן המפריד 0x00
- בתים 28-31: מספר ה-inode ששייך לקובץ